

Integrating ProMBA into the CIA-toolchain for MBA Deobfuscation

Written by Jeffrey Kim

April 13, 2025

Abstract

This paper details an exhaustive technical approach for integrating the MBA deobfuscation system **ProMBA**, which utilizes program synthesis and term rewriting techniques, into the full workflow of the CIA-toolchain (Chosen Instruction Attack toolchain). Based on the architecture and documentation provided at <https://github.com/chosen-instruction-attack/CIA-doc-en>, we describe how ProMBA can be used to simplify complex mixed boolean-arithmetic (MBA) expressions extracted from obfuscated virtual handlers. The integration is designed to improve success rates, provide more comprehensible mapping rules, and enable more accurate exploit generation and binary analysis.

1 Introduction

VM-based obfuscators often transform simple operations into intricate MBA expressions to hinder reverse engineering. The CIA-toolchain aims to reverse such obfuscators by using chosen instruction attacks and dynamic analysis to extract the hidden semantics of virtualized instructions. Meanwhile, ProMBA is a state-of-the-art tool for simplifying MBA expressions by combining program synthesis, term rewriting, and dynamic rule learning. In this whitepaper, we propose a deep integration of ProMBA into the CIA-toolchain.

By integrating ProMBA, the CIA workflow can simplify the complex symbolic formulas extracted from virtual handlers into concise, human-readable forms. This not only makes the mapping rules easier to verify, but also facilitates downstream tasks such as devirtualization and exploit synthesis.

2 CIA-toolchain Workflow Overview

The CIA-toolchain is designed to extract the semantics of obfuscated virtual instructions. Its major stages include:

1. **Knowledge-Leaking Program Construction:** Construct a program P with chosen instructions that are framed by **anchor** instructions. These anchors remain unvirtualized and serve as markers.

2. **Obfuscation and Execution Tracing:** Feed P into the obfuscator to produce $O(P)$. Execute the obfuscated binary under instrumentation (e.g., using Intel PIN) to record a trace of the instructions executed between the anchor points.
3. **Trace Slicing and IR Lifting:** Use slicing techniques to isolate the virtual handler corresponding to the chosen instruction, and lift the sliced trace into an intermediate representation (IR).
4. **Symbolic Execution:** Perform symbolic execution on the IR to generate a symbolic formula describing the handler's effect.
5. **Knowledge Extraction:** Verify the extracted formula against the expected semantics, and record a mapping rule that links the virtual handler to the original instruction.

Figure 1 illustrates the baseline CIA-toolchain workflow.



Figure 1: CIA-toolchain Workflow (Baseline).

3 Overview of ProMBA

ProMBA is designed to simplify overly complex MBA expressions. Its key techniques include:

- **Program Synthesis:** Generating candidate simpler expressions by testing input/output behavior.
- **Term Rewriting:** Applying algebraic rewrite rules, including those learned on-the-fly, to locally simplify expressions.
- **Linear and Polynomial Decomposition:** Preprocessing an input expression to separate linear parts from more complex portions.

ProMBA has been shown to reduce obfuscated expressions (often having hundreds or thousands of AST nodes) down to near-original, concise expressions. For example, it can simplify a 1120-node expression to a 3-node expression representing a simple subtraction ($x - y$).

4 Integration Points: Where to Invoke ProMBA in CIA

We propose integrating ProMBA into the CIA workflow in the following stage:

1. **After IR Lifting and Symbolic Execution:** Once the virtual handler’s trace is lifted to IR and symbolically executed, a complex symbolic formula is produced that represents the semantics of the chosen instruction.
2. **Invoking ProMBA:** Prior to (or concurrently with) complete SMT verification, export the symbolic formula to a C expression that serves as input for ProMBA. ProMBA then simplifies the obfuscated MBA expression.
3. **Verification and Rule Recording:** Use an SMT solver (such as Z3) to verify that the simplified expression is equivalent to the original symbolic formula. Once verified, record the mapping rule.

4.1 Data Exchange and Interface Design

- **Format Conversion:** Convert the IR or symbolic formula (from angr/Claripy or Z3) into a C language file containing a function (e.g., `target_expr`) that computes the obfuscated operation. This function can be auto-generated using a template.
- **ProMBA Invocation:** CIA-toolchain calls ProMBA as an external process, e.g., via a command-line call:

```
./promba -timeout 60 target_expr.c
```

- **Result Parsing:** Capture ProMBA’s output (the simplified expression and meta-data such as size reduction) and parse it back into the CIA-toolchain.

4.2 Integrated Workflow Diagram

Figure 2 illustrates the proposed CIA+ProMBA integration.



Figure 2: CIA-toolchain with ProMBA Integration. Blue blocks represent the new ProMBA processing step.

5 Example Use Cases

5.1 Simplification of Virtual Handler MBA Expression

1. CIA constructs a chosen-instruction program that contains a target arithmetic operation (e.g., an `ADD` instruction).
2. The program is obfuscated by a commercial protector, producing a virtual handler rich in MBA transformations.
3. CIA extracts the virtual handler trace, performs IR lifting and symbolic execution, and produces a complex formula (with hundreds of operations).
4. The IR formula is exported to a C file and passed to ProMBA. ProMBA simplifies the formula to a concise form (e.g., `r = x + y`).
5. CIA verifies the equivalence using an SMT solver and then records the mapping rule.

5.2 Pre-Processing for Exploit Generation

1. A malware sample uses MBA obfuscation to hide a key check in a critical function.
2. Using CIA, the relevant handler is isolated and symbolically executed, yielding a highly complex condition.
3. ProMBA simplifies this condition to a concise equality (e.g., `if (x == secret_key)`).
4. The simplified expression is then fed into the exploit generator, making automated input synthesis far more tractable.

6 Advantages of the Integrated Approach

- **Enhanced Simplification:** ProMBA converts highly complex MBA formulas into minimal expressions, aiding in clear interpretation.
- **Increased Success Rate:** Complicated virtual handlers that would cause SMT timeouts are more likely to be correctly simplified.
- **Improved Automation:** The system can automatically produce mapping rules that are both verified and human-readable, facilitating automated devirtualization and exploit synthesis.
- **Scalability for Future Obfuscation Techniques:** Since ProMBA learns new rewrite rules on-the-fly, the integrated system is more adaptable to novel or stronger MBA obfuscation patterns.

7 Challenges and Limitations

- **Performance Overhead:** Synthesizing and SMT checking may introduce delays, particularly for extremely large expressions.
- **Data Format Compatibility:** Converting IR expressions to a ProMBA-friendly C format requires robust translation mechanisms.
- **Integration Complexity:** Ensuring smooth communication between CIA (typically implemented in Python/angr) and ProMBA (implemented in OCaml/C++) entails careful API and error handling design.
- **Limited Scope:** ProMBA is most effective on combinatorial arithmetic logic; if the handler involves memory indirection or complex control flow beyond arithmetic, additional preprocessing is necessary.

8 Conclusion

Integrating ProMBA into the CIA-toolchain provides a synergistic enhancement for reverse-engineering obfuscated binaries. CIA extracts complex virtual handler logic through dynamic analysis and symbolic execution, while ProMBA simplifies these expressions to their core arithmetic or logical operations. The resulting system improves mapping rule extraction, enhances automated exploit generation, and offers a more comprehensive solution against advanced VM-based obfuscators. Although challenges such as performance and integration complexity exist, proper interface design, error handling, and potential parallelization can mitigate these issues. Future work may focus on deeper integration (e.g., embedding ProMBA’s rules into symbolic execution) and extending support to additional obfuscation techniques, thereby reinforcing that no single obfuscation technique is immune to a multi-layered, integrated reverse-engineering attack.

References:

(References in the text are denoted by numbers and page/line indicators, which correspond to the original ProMBA and CIA literature.)